

# Week 2- Algorithm Correctness and Mathematical Induction

Instructors: Dr. Fatima Adamu-Fika  
and Ms. Onyinye Okpoko

# Objectives

- Understand the concept of algorithm correctness.
- Learn how mathematical induction can prove correctness.
- Apply loop invariants to verify iterative algorithms.
- Students will gain skills to verify the correctness of simple algorithms.

# What is algorithm correctness?

Algorithm correctness ensures that an algorithm produces the expected results for all possible inputs.

However, it is not enough for the algorithm to produce correct results, it must also be ensured that it terminates. Hence, there are two types of correctness: partial and total correctness.

# Partial correctness

- Partial correctness ensures that the algorithm produces the correct result if it terminates, but it doesn't guarantee termination.
- Partial correctness is especially important when we're only concerned with an algorithm's logical accuracy and not whether it reaches an endpoint. For example, consider algorithms that run indefinitely (e.g., monitoring systems); they must be partially correct for their results to be reliable, even though they aren't designed to terminate.
- Example: A program that searches for a number in a list. If it finds the number, it returns the correct position but may sometimes get stuck in a loop. This only achieves partial correctness.

# Total correctness

- Total correctness, on the other hand, combines both: the algorithm produces the correct result and completes in a finite number of steps.
- Total correctness is the ideal scenario and is generally what we strive for in well-defined algorithms, where we expect both a correct output and termination. It's essential for most applications because an incomplete algorithm is often of limited value.
- Example: A program that adds all numbers from 1 to  $n$  always produces the correct sum and finishes for any valid input  $n$ , meaning it has total correctness

# Significance of algorithm correctness

- Ensures reliability, making software safe and predictable.
- Avoids errors in critical areas, such as finance, healthcare, and safety.

Real world application: A banking algorithm for calculating interest must be correct to prevent financial discrepancies.

Correctness is crucial in fields like banking, healthcare, and safety-critical systems.

Example: Ensuring that control algorithms in aviation software maintain safe and stable flight paths.

# Introduction to Mathematical Induction

- Definition: Mathematical induction is a method for proving statements or formulas for all natural numbers.

Steps in Mathematical Induction:

- Base Case: Prove the statement for the first value (e.g.,  $n=1$ ).
- Inductive Step: Assume the statement holds for  $n$  and prove it for  $n+1$ .

Purpose: Induction allows us to show that a property holds for all cases by proving it for the base and the next steps.

# Examples of mathematical induction

Prove that the sum of the first  $n$  positive integers is  $S = (n(n+1))/2$ .

- Base Case:

For  $n=1$ ,  $S = (1(1+1))/2$ .

- Inductive Step:

Inductive Hypothesis Assume it holds for  $n=k$

RTP: Show it holds for  $n= k+1$ .

- Outcome: This example demonstrates the step-by- step process of proving a formula using induction

# Strong Induction

Strong induction, also known as the second principle of mathematical induction, is a proof technique used to establish a property for all natural numbers. Unlike simple induction, strong induction assumes the property holds for all previous values up to a given point.

# How it works

- Base Case: Prove the property is true for the initial value(s) (often  $n = 0$  or  $n = 1$ ).
- Inductive Step: Assume the property holds for all values up to  $n$  (i.e.,  $P(1), P(2), \dots P(n)$ ). Then, prove the property for  $n + 1$ .

Strong induction is particularly useful when each case depends on multiple preceding cases. **It is therefore ideal for proving recursive algorithms as it allows us to validate that a recursive solution remains correct as it builds upon smaller instances of itself.**

# Proving Algorithm Correctness with Induction

Process:

- Define the Statement: Clearly state what the algorithm is intended to accomplish.
- Base Case: Verify the algorithm's correctness for the simplest input.
- Inductive Step: Show that if the algorithm works for  $n=k$ , it also works for  $n=k+1$ .
- Example: Use a simple recursive function, such as calculating the factorial of  $n$ , to illustrate how induction can prove correctness.

# Introduction to Loop Invariants

- Definition: A loop invariant is a condition that holds true before and after each iteration of a loop.

The purpose of a loop invariant is to prove the correctness of iterative algorithms by showing that the loop progresses toward the goal.

# Characteristics of a Good Loop Invariant

A good loop invariant should satisfy three properties:

- Initialisation: The invariant is true before the start of the loop.
- Maintenance: The invariant remains true after each iteration.
- Termination: When the loop ends, the invariant helps demonstrate that the algorithm has achieved its goal.

Another desirable characteristic is **clarity**. That is, invariant should be clear and related to the algorithm's goal.

A loop invariant is usually placed at the beginning of the loop. A well-chosen loop invariant is useful both when designing, testing, and modifying code. It also serves as documentation and can be the foundation of correctness proof.

# Application of induction to iterative algorithms: Example 1

Refer to Iterative POW2(n) from week 1

Claim:

For all natural numbers, POW2(n) returns  $2^n$ .

Let's find a pattern as the program variables in POW2(n) change from one iteration of the loop to the next:

$$i_0 = 0, i_1 = 1, i_2 = 2, i_3 = 3, \dots$$

$$\text{Thus, } i_k = k$$

Also

$$\begin{aligned} power_0 &= 1 = 2^0, power_1 = 1 = 2^1, power_2 = 1 = 2^2, \\ power_3 &= 1 = 2^3, \dots \end{aligned}$$

$$\text{Thus, } power_k = 2^k$$

## The Loop Invariants of POW2(n)

For all natural numbers  $k \geq 0$ ,  $i_k = k$ , and  $power_k = 2^k$ .  
The proof is by induction on  $k$ .

Base Step:

When  $n = 0$

$k = 0$ , since  $i_0 = 0$  [From the loop invariants we know that  $i_k = k$ ].

So,  $power_0 = 1 = 2^0$ .

Inductive Step:

Induction hypothesis:

Now suppose that  $k \geq 0$ ,  $k \geq 0$ ,  $i_k = k$ , and  $power_k = 2^k$ .

RTP:

$$i_{k+1} = k + 1, \text{ and } power_{k+1} = 2^{k+1}.$$

Proof:

$$\begin{aligned} i_{k+1} &= i_k + 1 \\ &= k + 1 \quad \text{[by induction hypothesis]} \end{aligned}$$

$$\begin{aligned} power_{k+1} &= 2 \times power_k \\ &= 2 \times 2^k \quad \text{[by induction hypothesis]} \\ &= 2^{k+1} \end{aligned}$$

Conclusion: The loop invariant holds.

Correctness proof for POW2( $n$ ) terminating with the correct result:

Claim:

POW2( $n$ ) terminates with power containing  $2^n$

Termination:

Since  $i_{k+1} = i_k + 1$ , eventually  $i$  will be equal  $n$  and the loop will terminate.

Suppose this happens after  $t$  iterations:

Since  $i_t = n$  and  $i_t = t$  [ $\because i_k = k$ ], we can conclude that  $t = n$  [ $\because i_t = n = t$ ].

Results :

By the loop invariant,  $power_t = 2^t = 2^n$  [ $\because t = n$ ].

# Application of induction to iterative algorithms: Example 2

Refer to Iterative EVENSUM(n) from week 1

Claim:

For all  $n \geq 1$ , EVENSUM(n) returns  $\sum_{i=1}^n (2i - 2)$

To find the loop invariant, let's find a pattern of its program variables within the loop, through each execution of the loop

$$i_0 = 1 = 0 + 1, i_1 = 2 = 1 + 1, i_2 = 3 = 2 + 1, i_3 = 4 = 3 + 1, \dots$$

$$\text{Thus, } i_k = k + 1$$

Also

$$even_0 = 0$$

$$even_1 = 0 = even_0 + (2i_0 - 2) = even_0 + [2(1) - 2]$$

$$even_2 = 2 = even_1 + (2i_1 - 2) = even_1 + [2(2) - 2],$$

$$even_3 = 6 = even_2 + (2i_2 - 2) = even_2 + [2(3) - 2], \dots$$

$$\text{Thus, } even_k = even_{k-1} + (2i_{k-1} - 2) = even_{k-1} + [2(k) - 2]$$

Loop invariant for EVENSUM(n)

For all natural numbers  $k \geq 0$ ,  $i_k = k + 1$  and  $even_k = \sum_{i=1}^k (2i - 2)$ .

The proof is by induction on  $k$ .

Basis:

When  $k = 0$ :

$$i_0 = 1 \text{ and } even_0 = 0$$

Inductive step:

Induction hypothesis:

$$\text{Assume that } k \geq 0, i_k = k + 1 \text{ and } even_k = \sum_{i=1}^k (2i - 2)$$

RTP:

$$i_{k+1} = k + 2 \text{ and } even_{k+1} = \sum_{i=1}^{k+1} (2i - 2).$$

Proof:

$$\begin{aligned} i_{k+1} &= i_k + 1 \\ &= k + 1 + 1 \\ &= k + 2 \end{aligned}$$

$$\begin{aligned} even_{k+1} &= even_k + (2i_k - 2) \\ &= \sum_{i=1}^k (2i - 2) + (2(k + 1) - 2) \text{ [by induction hypothesis : } k \\ &\geq 0, i_k = k + 1 \text{ and } even_k = \sum_{i=1}^k (2i - 2)] \\ &= \sum_{i=1}^{k+1} (2i - 2) \end{aligned}$$

Conclusion: The loop invariant holds.

# Correctness proof for EVENSUM(n) terminating with the correct result

Claim:

EVENSUM(n) terminates with containing  $\sum_{i=1}^n (2i - 2)$

Termination :

Since  $i_{k+1} = i_k + 1$ , eventually  $i$  will equal  $n + 1$  and the loop will terminate. Assume this happens after  $t$  iterations.

Since  $i_t = n + 1$  and  $i_t = t + 1$  [ $\because i_k = k + 1$ ], we can conclude that  $t = n$

[ $\because i_t = n + 1 = t + 1$ ].

Results : By the loop invariant,

$$even_t = \sum_{i=1}^t (2i - 2) = \sum_{i=1}^n (2i - 2) [\because t = n].$$

Note that the proof by induction is the initialisation and maintenance phase of the loop invariant.

# Application of strong induction to recursive algorithms: Example 1

Refer to Recursive POW2(n) from week 1

Claim:

For all  $n \geq 0$ , POW2(n) returns  $2^n$ .

Basis step:

- For  $n = 0$ , POW2(0) returns  $1 = 2^0$  as claimed.
- For  $n = 0$ , POW2(0) returns  $1 = 2^0$  as claimed.
- For  $n = 0$ , POW2(0) returns  $1 = 2^0$  as claimed.

Inductive Step:

- Induction hypothesis:

Assume that  $k \geq 2$  and POW2(k) returns  $2^k$  for values  $0, 1, 2, \dots, k$ .

RTP:

POW2( $k + 1$ ) returns  $2^{k+1}$

Proof:

What does POW2( $k + 1$ ) actually return?

$$2 \times \text{POW2}(k)$$

$$= 2 \times 2^k \quad [\text{by induction hypothesis}]$$

$$= 2^{k+1}$$

Conclusion: Therefore, for all  $n \geq 0$ , POW2( $n$ ) returns  $2^n$ .

# Application of strong induction to recursive algorithms: Example 2

Refer to Recursive EVENSUM( $n$ ) from week 1

Claim:

For all  $n \geq 1$ , EVENSUM( $n$ ) returns  $\sum_{i=1}^n (2i - 2)$

Base Step:

For  $n = 1$ , EVENSUM(1) returns 0 as claimed.

For  $n = 2$ , EVENSUM(2) returns 2 as claimed.

Inductive Step:

Induction hypothesis:

Assume that  $k \geq 2$  and EVENSUM( $k$ ) returns  $\sum_{i=1}^k (2i - 2)$  for all numbers  $0, 1, 2, \dots, k$

RTP:

EVENSUM(k+1) returns  $\sum_{i=1}^{k+1} (2i - 2)$

Proof:

What does EVENSUM(k + 1) actually return?

$$\text{EVENSUM}(k) + (2(k + 1) - 2)$$

$$= \sum_{i=1}^k (2i - 2) + (2(k + 1) - 2)$$

$$= \sum_{i=1}^{k+1} (2i - 2)$$

Conclusion : Therefore, for all  $n \geq 1$ , EVENSUM(n) returns  $\sum_{i=1}^n (2i - 2)$ .

# Activities

- Activity 1: Practice proving correctness using induction. o Example Problem: Prove the correctness of a recursive algorithm (e.g., recursive function calculating the  $n^{th}$  factorial).
- Activity 2: Identify loop invariants in simple iterative algorithms. Example Problem: Verify the invariant for an algorithm that finds the maximum element in an array.

# Summary

In this topic we have:

- defined algorithm correctness and discussed its importance.
- learned mathematical induction and loop invariants as tools for proving correctness.
- practiced applying these concepts to simple algorithms.

# Further tasks

- Practice Problems: Practice proving correctness using induction and loop invariants through personal research and online materials.
- Preparation: Think of other real-life examples where correctness would be essential.